

Credit Name: CSE 3130 Object-Oriented Programming 2
Assignment Name: Vehicle, Car, Truck, Minivan

Understanding the Problem

How did you approach understanding the challenge?

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

I sort of understood the problem. The instructions in the textbook did seem rather vague. At the time, I didn't see the instance variables we were supposed to implement according to D2L, so I decided to make my own plan.

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding?

How did you decide on the tools, data structures, or algorithms to use?

I figured I would recycle most of the code from the UEmployee and SalesCenter projects. Because I initially didn't see some of the information in D2L, I came up with the idea to make multiple methods in the Vehicle class that would basically simulate driving one of the vehicles. I also planned on making an abstract variable that would simulate the sound of a vehicle honking, and it changed based on which vehicle was chosen. I ended up saving most of the code in the final project.

Implementation

Did you write the code in small pieces or attempt the entire solution at once?

How did you test your solution along the way to make sure it was working?

I started by following the instructions in the textbook and made the Vehicle class which, at the time, only included instance variables for the vehicle name and its cost.

```
private String name;  
private double cost;  
  
public Vehicle(String n, double c)  
{  
    name = n;  
    cost = c;  
}
```

I then coded the methods for simulating being in a vehicle. Along with the ones in the image below, I also had methods for starting and stopping the engine of the vehicle.

```

public String drive()
{
    return("Vroom Vroom!");
}

public String stop()
{
    return("Screech!");
}

public abstract String honk();

```

After that, I made the Car, Truck, and Minivan classes. Along with them, I coded the abstract method that would change the sound of the car horn. I was inspired to make the honk method while researching how abstract methods were used. As mentioned earlier, I wasn't aware of the fuelEconomyCity, fuelEconomyHwy, seatingCapacity, and cargoVolume instance variables when I first made them.

```

public class Car extends Vehicle
{
    public Car(String n, double c, int city, int hwy, int seatCap, String carVol)
    {
        super(n, c, city, hwy, seatCap, carVol);
    }

    public String honk()
    {
        return("Beep Beep!");
    }
}

```

```

public class Truck extends Vehicle
{
    public Truck(String n, double c, int city, int hwy, int seatCap, String carVol)
    {
        super(n, c, city, hwy, seatCap, carVol);
    }

    public String honk()
    {
        return("Honk Honk!");
    }
}

```

```

public class Minivan extends Vehicle
{
    public Minivan(String n, double c, int city, int hwy, int seatCap, String carVol)
    {
        super(n, c, city, hwy, seatCap, carVol);
    }

    public String honk()
    {
        return("Wah Wah!");
    }
}

```

I then made a class called Drive in order to test everything. In it, I basically recycled the code from UEmployee for the main interface as well as the choice to simulate driving the vehicle.

```

do
{
    System.out.println("(D)rive \\ (N)ame and Price \\ (S)eat Capacity \\ (F)uel Economy \\ (C)argo Volume \\ (Q)uit?");
    System.out.println("Enter Your Choice: ");
    choice = input.next();

    if (!choice.equalsIgnoreCase("Q"))
    {
        System.out.println("Enter Vehicle Number (1, 2, 3): ");

        vNum = input.nextInt();

        switch (vNum)
        {
            case 1: v = v1; break;
            case 2: v = v2; break;
            case 3: v = v3; break;
        }
    }
}

```

```

if (choice.equalsIgnoreCase("D"))
{
    do
    {
        System.out.println("Do You Want to (D)rive, (S)top, (H)onk, or (T)urn Off Engine?");
        System.out.println("Enter Your Choice: ");
        action = input.next();

        if (action.equalsIgnoreCase("H"))
        {
            System.out.println(v.honk());
        }

        else if (action.equalsIgnoreCase("D"))
        {
            System.out.println(v.drive());
        }

        else if (action.equalsIgnoreCase("S"))
        {
            System.out.println(v.stop());
        }

    } while(!action.equalsIgnoreCase("T"));
}

```

At that point, I thought I was done. It was only after I finished the Account mastery project and I double checked D2L when I noticed my mistake. I ended up going back to my code and adding the missing instance variables in the vehicle class as well as their respective methods.

```
public String getFuelEco()
{
    return("The " + name + " Has a Fuel Economy of " + fuelEconomyCity + " MPG in a City and " + fuelEconomyHwy + " MPG on a Highway");
}

public String getSeatCap()
{
    return("The Seating Capacity of the " + name + " is " + seatingCapacity);
}

public String getCarVol()
{
    return("The Cargo Volume Ranges From " + cargoVolume + " Cubic Feet");
}
```

I then went back to the Drive class to add the choice to check the information that correlates to the instance variables.

```
else if (choice.equalsIgnoreCase("S"))
{
    System.out.println(v.getSeatCap());
}

else if (choice.equalsIgnoreCase("F"))
{
    System.out.println(v.getFuelEco());
}

else if (choice.equalsIgnoreCase("C"))
{
    System.out.println(v.getCarVol());
}
```

Overcoming Challenges

What part of the problem was the most difficult for you?

How did you handle moments when you felt unsure of what to do next?

There wasn't really any part I felt confused besides my initial reaction to the question. Everything made more sense when I looked back at D2L.

Learning

Was there anything you learned that you think will help you with future challenges?

My understanding of inheritance and polymorphism has improved.